

Ryan Fanning -

Documentation of Design Patterns Used

1. Inheritance – This was used in `game_object.py` -> `player.py` / `enemy.py` / `item.py`. The purpose of inheritance is to reduce duplication within code and is used as a base class to share functionality from different code files.
2. Polymorphism – This was used as `updateAll(self, dt)` and `renderAll(self, screen)` in `world.py`. `GameWorld` calls `update` and `render` on all objects without knowing their types. This makes the code more flexible as multiple different types can be used.
3. Factory pattern – This was used as `EnemySpawner` in `enemy_spawner.py`. It dynamically creates enemy objects at runtime using creation logic.
4. GameLoop – This was used in `game.py`. The purpose of `gameloop` is to continuously update the objects, handle input, check for collisions, render the game, keep the game smooth and to bring the game alive by running it.
5. Singular Responsibility – Each class has a single job and this helps code to be easier read and more maintainable.

Performance Analysis

The Dino Runner game works by running a loop that keeps the game updating and showing everything on the screen. Every frame the game updates all the objects, like the player and the enemies, and moves them based on their speed and gravity. This makes the game feel smooth and allows the player to jump and crouch without delays. The game also draws all the objects every frame, which is fast because it uses simple shapes instead of complicated pictures or graphics. Collisions are checked by seeing if the player touches any enemies. This is easy to do and works well with only a few enemies, but if there were too many objects at once it could slow the game down. Each object only keeps basic information like position, size, and movement, so the game does not use much memory. The clock in `pygame` keeps the speed of the game consistent so everything moves at the right pace no matter what computer it is running on. Overall the game runs smoothly, feels responsive, and does not lag. Even though the game works well now, it could be made faster if enemy objects were reused instead of being created and destroyed each time, or if collisions were checked in a smarter way. For this small game, the current setup works fine and is enough to show that the game engine runs correctly and can handle real-time gameplay.

Test Samples / Known Bugs

The only test sample is to play the game and dodge the obstacles or to crash into one. I have no known bugs as it is a simple game.